RELATÓRIO DE CONCLUSÃO

Gerar carga e observar comportamento

Projeto: Seminário Técnico — Grafana

Curso: Sistemas de Informação — UniSales

Ambiente validado: Node.js + Prometheus + Grafana + Docker Compose

Status: CONCLUÍDA

Data de validação: 19/10/2026

1. Objetivo da entrega

A quarta tarefa teve como objetivo produzir tráfego controlado contra a API e observar se as métricas instrumentadas reagiam às alterações de volume, rotas e códigos HTTP. O teste também deveria demonstrar um erro 500 de forma intencional.

2. Carga executada

Foi utilizado PowerShell com Invoke-WebRequest, evitando a necessidade de uma ferramenta adicional de carga nesta fase. Foram realizadas 100 requisições distribuídas da seguinte forma:

```
50 chamadas -> GET /api/test
30 chamadas -> GET /api/users
20 chamadas -> GET /api/error (HTTP 500 esperado)
```

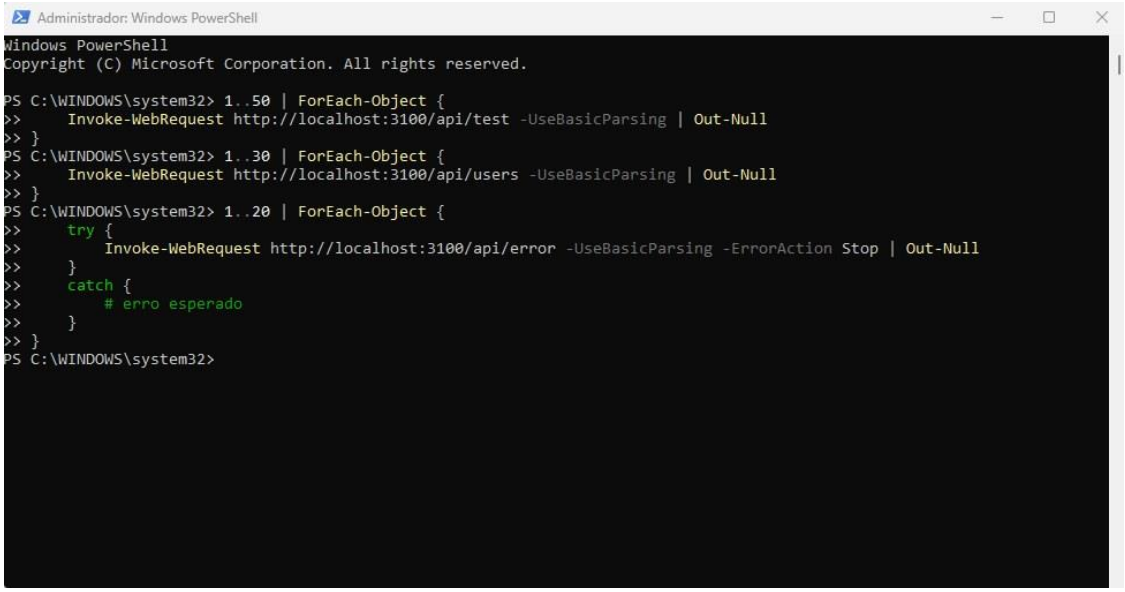


Figura 1 – Geração de carga controlada com loops no PowerShell.
Fonte: evidência produzida durante a validação prática do projeto (2026).

3. Comportamento observado

Após o teste e os scrapes seguintes do Prometheus, o dashboard refletiu o aumento da taxa de requisições. O painel de códigos HTTP passou a apresentar HTTP 500 e o painel de rotas passou a incluir /api/error e /api/test. A API permaneceu disponível durante o experimento.

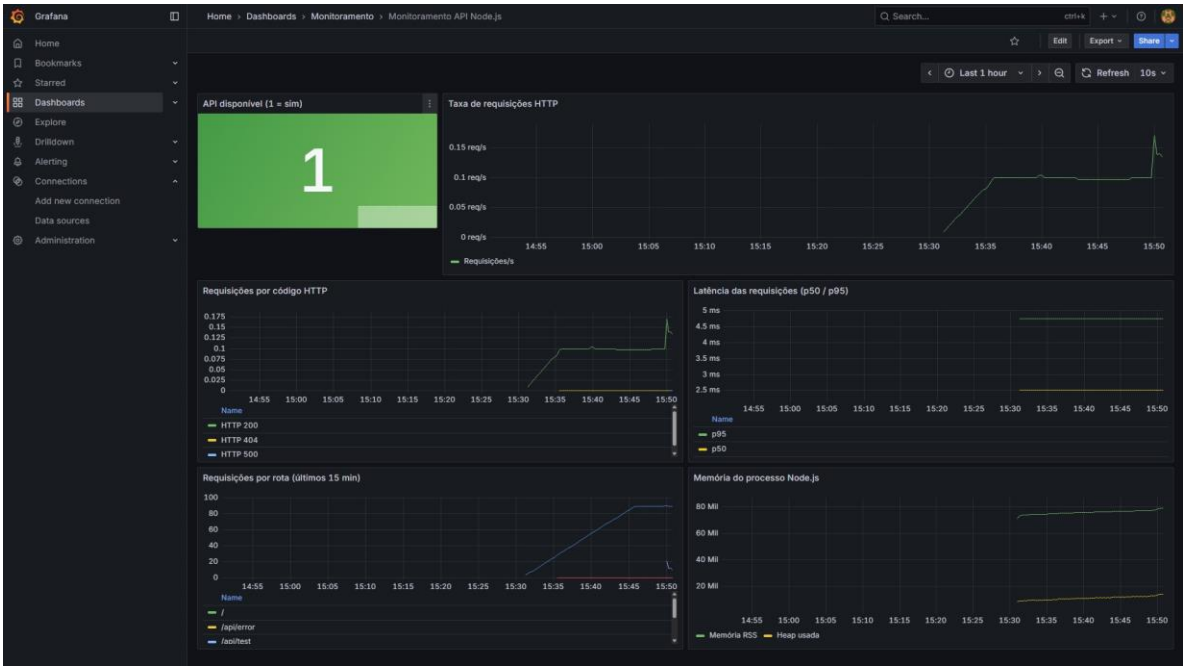


Figura 2 – Dashboard após a carga, com crescimento de tráfego, HTTP 500 e novas rotas.
Fonte: evidência produzida durante a validação prática do projeto (2026).

4. O que o teste comprovou

Critério	Resultado	Status
Variação de volume	Taxa de requisições aumentou após a carga.	**Aprovado**
Rota /api/test	Chamadas apareceram no painel por rota.	**Aprovado**
Rota /api/error	Chamadas apareceram no painel por rota.	**Aprovado**
HTTP 500	Série passou a ser exibida por código HTTP.	**Aprovado**
Disponibilidade	Indicador up permaneceu em 1.	**Aprovado**
Latência e memória	Painéis continuaram recebendo dados durante o teste.	**Aprovado**

5. Alteração em relação ao planejamento original

Planejamento original	Implementação validada
O planejamento previa gerar carga/testes, sem determinar uma ferramenta obrigatória.	Foi adotado PowerShell com Invoke-WebRequest para uma carga simples e reproduzível: 50 chamadas de teste, 30 de usuários e 20 de erro.

Impacto da mudança: A solução é suficiente para demonstrar o comportamento das métricas nesta fase, sem introduzir uma dependência adicional. Ferramentas como k6 ou Locust podem ser usadas futuramente se for necessário um teste de carga mais intenso.

6. Conclusão

A carga controlada comprovou que as métricas respondem ao comportamento real da aplicação. O teste validou a relação entre requisições produzidas, coleta pelo Prometheus e atualização do dashboard Grafana, encerrando a Fase 3 com evidência prática.